
Functional Programming

Functional programming (FP) is a big topic, not to be learned in twenty-one days¹ or in a single chapter of a book. Nevertheless, you can reach a first level of effectiveness using lazy and recursive techniques in Clojure fairly quickly, and that is what we'll accomplish this chapter.

Here's how we'll do that:

- In [Section 4.1, *Functional Programming Concepts*, on page 85](#), you'll get a quick overview of FP terms and concepts. This section also introduces the “Six Rules of Clojure FP” that we will refer to throughout the chapter.
- In [Section 4.2, *How to Be Lazy*, on page 90](#), you'll experience the power of lazy sequences. You will create several implementations of the Fibonacci numbers, starting with a terrible approach and improving it to an elegant, lazy solution.
- As cool as lazy sequences are, you rarely need to work with them directly. In [Section 4.3, *Lazier Than Lazy*, on page 98](#), you'll see how to rethink problems so that they can be solved directly using the sequence library described in [Chapter 3, *Unifying Data with Sequences*, on page 55](#).
- And in [Section 4.4, *Recursion Revisited*, on page 103](#), we'll explore some advanced issues. Some programmers will never need the techniques discussed here. If you are new to FP, it is OK to skip this section.

4.1 Functional Programming Concepts

Functional programming leads to code that is easier to write, read, test, and reuse. Here's how it works.

1. <http://norvig.com/21-days.html>

Pure Functions

Programs are built out of *pure functions*. A pure function has no *side effects*; that is, it does not depend on anything but its arguments, and its only influence on the outside world is through its return value.

Mathematical functions are pure functions. Two plus two is four, no matter where and when you ask. Also, asking doesn't *do* anything other than return the answer.

Program output is decidedly *impure*. For example, when you `println`, you change the outside world by pushing data onto an output stream. Also, the results of `println` depend on state outside the function: the standard output stream might be redirected, closed, or broken.

If you start writing pure functions, you will quickly realize that pure functions and *immutable* data go hand in hand. Consider the following mystery function:

```
(defn mystery [input]
  (if input data-1 data-2))
```

If `mystery` is a pure function, then regardless of what it does, `data-1` and `data-2` have to be immutable! Otherwise, changes to the data would cause the function to return different values for the same input.

A single piece of mutable data can ruin the game, rendering an entire call chain of functions impure. So, once you make a commitment to writing pure functions, you end up using immutable data in large sections of your application.

Persistent Data Structures

Immutable data is critical to Clojure's approach to both FP and state. On the FP side, pure functions cannot have side effects, such as updating the state of a mutable object. On the state side, Clojure's reference types require immutable data structures to implement their concurrency guarantees.

The fly in the ointment is performance. When all data is immutable, "update" translates into "create a copy of the original data, plus my changes." This will use up memory quickly! Imagine that you have an address book that takes up 5MB of memory. Then, you make five small updates. With a mutable address book, you are still consuming about 5MB of memory. But if you have to copy the whole address book for each update, then an immutable version would balloon to 25MB!

Clojure's data structures do not take this naive “copy everything” approach. Instead, all Clojure data structures are *persistent*. In this context, persistent means that the data structures preserve old copies of themselves by efficiently *sharing structure* between older and newer versions.

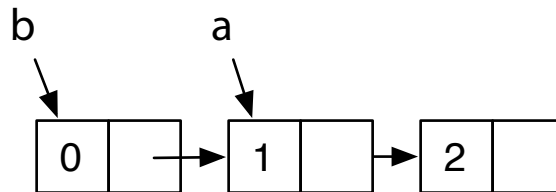
Structural sharing is easiest to visualize with a list. Consider list *a* with two elements:

```
(def a '(1 2))
-> #'user/a
```

Then, from *a* you can create *b* with an additional element added:

```
(def b (cons 0 a))
-> #'user/b
```

b is able to reuse all of *a*'s structure, rather than having its own private copy:



All of Clojure's data structures share structure where possible. For structures other than simple lists, the mechanics are more complex, of course. If you are interested in the details, check out the following articles:

- “Ideal Hash Trees”² by Phil Bagwell
- “Understanding Clojure's PersistentVector Implementation”³ by Karl Krukow

Laziness and Recursion

Functional programs make heavy use of *recursion* and *laziness*. A recursion occurs when a function calls itself, either directly or indirectly. With laziness, an expression's evaluation is postponed until it is actually needed. Evaluating a lazy expression is called *realizing* the expression.

In Clojure, functions and expressions are not lazy. However, sequences *are* generally lazy. Because so much Clojure programming is sequence manipulation, you get many of the benefits of a fully lazy language. In particular, you can build complex expressions using lazy sequences and then pay only for the elements you actually need.

2. <http://lampwww.epfl.ch/papers/idealhashtrees.pdf>

3. <http://tinyurl.com/clojure-persistent-vector>

Lazy techniques imply pure functions. You never have to worry about when to call a pure function, since it always returns the same thing. Impure functions, on the other hand, do not play well with lazy techniques. As a programmer, you must explicitly control when an impure function is called, because if you call it at some other time, it may behave differently!

Referential Transparency

Laziness depends on the ability to replace a function call with its result at any time. Functions that have this ability are called *referentially transparent*, because calls to such functions can be replaced without affecting the behavior of the program. In addition to laziness, referentially transparent functions can also benefit from the following:

- *Memoization*, automatic caching of results
- Automatic *parallelization*, moving function evaluation to another processor or machine

Pure functions are referentially transparent *by definition*. Most other functions are *not* referentially transparent, and those that are must be proven safe by code review.

Benefits of FP

Well, that is a lot of terminology, and we promised it would make your code easier to write, read, test, and compose. Here's how.

You'll find functional code easier to *write* because the relevant information is right in front of you, in a function's argument list. You do not have to worry about global scope, session scope, application scope, or thread scope. Functional code is easier to *read* for exactly the same reason.

Code that is easier to read and write is going to be easier to test, but functional code brings an additional benefit for testing. As projects get large, it often takes a lot of effort to set up the right environment to execute a test. This is much less of a problem with functional code, because there *is no relevant environment* beyond the function's arguments.

Functional code improves *reuse*. To reuse code, you must be able to do the following:

- Find and understand a piece of useful code.
- Compose the reusable code with other code.

The readability of functional code helps you find and understand the functions you need, but the benefit for *composing* code is even more compelling.

Composability is a hard problem. For years programmers have used *encapsulation* to try to create composable code. Encapsulation creates a firewall, providing access to data only through a public API.

Encapsulation helps, but it is nowhere near enough. Even with encapsulated objects, there are far too many surprising interactions when you try to compose entire systems. The problem is those darn side effects. *Impure functions* violate encapsulation, because they let the outside world reach in (invisibly!) and change the behavior of your code. Pure functions, on the other hand, are truly encapsulated and composable. Put them anywhere you want in a system, and they will always behave in the same way.

The Six Rules

Although the benefits of FP are compelling, FP is a wholesale change from the imperative programming style that dominates much of the programming world today. Plus, Clojure takes a unique approach to FP that strikes a balance between academic purity and the reality of running well on the JVM. That means there is a lot to learn all at once. But fear not. If you are new to FP, the following “Six Rules of Clojure FP” will help you on your initial steps toward FP mastery, Clojure-style:

1. Avoid direct recursion. The JVM cannot optimize recursive calls, and Clojure programs that recurse will blow their stack.
2. Use `recur` when you are producing scalar values or small, fixed sequences. Clojure *will* optimize calls that use an explicit `recur`.
3. When producing large or variable-sized sequences, always be lazy. (Do *not* `recur`.) Then, your callers can consume just the part of the sequence they actually need.
4. Be careful not to realize more of a lazy sequence than you need.
5. Know the sequence library. You can often write code without using `recur` or the lazy APIs at all.
6. Subdivide. Divide even simple-seeming problems into smaller pieces, and you will often find solutions in the sequence library that lead to more general, reusable code.

Rules 5 and 6 are particularly important. If you are new to FP, you can translate these two rules to this: “Ignore this chapter and just use the techniques in [Chapter 3, Unifying Data with Sequences, on page 55](#) until you hit a wall.”

Like most rules, the six rules are guidelines, not absolutes. As you become comfortable with FP, you will find reasons to break them.

Now, let's get started writing functional code.

4.2 How to Be Lazy

Functional programs make great use of *recursive definitions*. A recursive definition consists of two parts:

- A *basis*, which explicitly enumerates some members of the sequence
- An *induction*, which provides rules for combining members of the sequence to produce additional members

Our challenge in this section is converting a recursive definition into working code. You might do this in several ways:

- A simple recursion, using a function that calls itself in some way to implement the induction step.
- A tail recursion, using a function only calling itself at the tail end of its execution. Tail recursion enables an important optimization.
- A lazy sequence that eliminates actual recursion and calculates a value later, when it is needed.

Choosing the right approach is important. Implementing a recursive definition poorly can lead to code that performs terribly, consumes all available stack and fails, consumes all available heap and fails, or does all of these. In Clojure, being lazy is often the right approach.

We will explore all of these approaches by applying them to the Fibonacci numbers. Named for the Italian mathematician Leonardo (Fibonacci) of Pisa (c.1170–c.1250), the Fibonacci numbers were actually known to Indian mathematicians as far back as 200 BC. The Fibonacci numbers have many interesting properties, and they crop up again and again in algorithms, data structures, and even biology.⁴ The Fibonacci numbers have a very simple recursive definition:

- Basis: F_0 , the zeroth Fibonacci number, is zero. F_1 , the first Fibonacci number, is one.
- Induction: For $n > 1$, F_n equals $F_{n-1} + F_{n-2}$.

4. http://en.wikipedia.org/wiki/Fibonacci_number

Using this definition, the first ten Fibonacci numbers are as follows:

(0 1 1 2 3 5 8 13 21 34)

Let's begin by implementing the Fibonacci using a simple recursion. The following Clojure function will return the *n*th Fibonacci number:

src/examples/functional.clj

```
Line 1 ; bad idea
2 (defn stack-consuming-fibo [n]
3   (cond
4     (= n 0) 0
5     (= n 1) 1
6     :else (+ (stack-consuming-fibo (- n 1))
7              (stack-consuming-fibo (- n 2)))))
```

Lines 4 and 5 define the basis, and line 6 defines the induction. The implementation is recursive because stack-consuming-fibo calls itself on lines 6 and 7.

Test that stack-consuming-fibo works correctly for small values of *n*:

```
(stack-consuming-fibo 9)
-> 34
```

Good so far, but there is a problem calculating larger Fibonacci numbers such as $F_{1000000}$:

```
(stack-consuming-fibo 1000000)
-> StackOverflowError clojure.lang.Numbers.minus (Numbers.java:1837)
```

Because of the recursion, each call to stack-consuming-fibo for $n > 1$ begets two more calls to stack-consuming-fibo. At the JVM level, these calls are translated into method calls, each of which allocates a data structure called a *stack frame*.⁵

The stack-consuming-fibo creates a depth of stack frames proportional to *n*, which quickly exhausts the JVM stack and causes the StackOverflowError shown earlier. (It also creates a total number of stack frames that is exponential in *n*, so its performance is terrible even when the stack does not overflow.)

Clojure function calls are designated as *stack-consuming* because they allocate stack frames that use up stack space. In Clojure, you should almost always avoid stack-consuming recursion as shown in stack-consuming-fibo.

5. For more on how the JVM manages its stack, see “Runtime Data Areas” at <http://tinyurl.com/jvm-spec-toc>.

Tail Recursion

Functional programs can solve the stack-usage problem with *tail recursion*. A tail-recursive function is still defined recursively, but the recursion must come at the tail, that is, at an expression that is a return value of the function. Languages can then perform *tail-call optimization* (TCO), converting tail recursions into iterations that do not consume the stack.

The stack-consuming-fibo definition of Fibonacci is not tail-recursive, because it calls add (+) *after* both calls to stack-consuming-fibo. To make fibo tail-recursive, you must create a function whose arguments carry enough information to move the induction forward, without any extra “after” work (like an addition) that would push the recursion out of the tail position. For fibo, such a function needs to know two Fibonacci numbers, plus an ordinal *n* that can count down to zero as new Fibonacci numbers are calculated. You can write tail-fibo thusly:

```
src/examples/functional.clj
Line 1 (defn tail-fibo [n]
2   (letfn [(fib
3           [current next n]
4             (if (zero? n)
5                 current
6                 (fib next (+ current next) (dec n))))]
7     (fib 0N 1N n)))
```

Line 2 introduces the `letfn` macro:

```
(letfn [fnspecs & body] ; fnspecs ==> [(fname [params*] exprs)+]
```

`letfn` is like `let` but is dedicated to letting local functions. Each function declared in a `letfn` can call itself or any other function in the same `letfn` block. Line 3 declares that `fib` has three arguments: the current Fibonacci, the next Fibonacci, and the number *n* of steps remaining.

Line 5 returns `current` when there are no steps remaining, and line 6 continues the calculation, decrementing the remaining steps by one. Finally, line 7 kicks off the recursion with the basis values 0 and 1, plus the ordinal *n* of the Fibonacci we are looking for.

`tail-fibo` works for small values of *n*:

```
(tail-fibo 9)
-> 34N
```

But even though it is tail-recursive, it still fails for large *n*:

```
(tail-fibo 1000000)
-> StackOverflowError java.lang.Integer.numberOfLeadingZeros (Integer.java:1054)
```

The problem here is the JVM. While functional languages such as Haskell can perform TCO, the JVM was not designed for functional languages. No language that runs directly on the JVM can perform automatic TCO.⁶

The absence of TCO is unfortunate but not a showstopper for functional programs. Clojure provides several pragmatic workarounds: explicit self-recursion with `recur`, lazy sequences, and explicit mutual recursion with trampoline.

Self-recursion with `recur`

One special case of recursion that *can* be optimized away on the JVM is a self-recursion. Fortunately, the tail-fibo is an example: it calls itself directly, not through some series of intermediate functions.

In Clojure, you can convert a function that tail-calls itself into an explicit self-recursion with `recur`. Using this approach, convert tail-fibo into recur-fibo:

```
src/examples/functional.clj
Line 1 ; better but not great
2 (defn recur-fibo [n]
3   (letfn [(fib
4             [current next n]
5             (if (zero? n)
6                 current
7                 (recur next (+ current next) (dec n))))]
8     (fib 0N 1N n)))
```

The critical difference between tail-fibo and recur-fibo is on line 7, where `recur` replaces the call to `fib`.

The recur-fibo will not consume stack as it calculates Fibonacci numbers and can calculate F_n for large n if you have the patience:

```
(recur-fibo 9)
-> 34N

(recur-fibo 1000000)
-> 195 ... 208,982 other digits ... 875N
```

The complete value of $F_{1000000}$ is included in the sample code at `output/f-1000000`.

The recur-fibo calculates one Fibonacci number. But what if you want several? Calling recur-fibo multiple times would be wasteful, since none of the work from

6. On today's JVMs, languages can provide automatic TCO for *some* kinds of recursion but not for *all*. Since there is no general solution, Clojure forces you to be explicit. When and if general TCO becomes widely supported on the JVM, Clojure will support it as well.

any call to `recur-fibo` is ever cached for the next call. But how many values should be cached? Which ones? These choices should be made by the *caller* of the function, not the implementer.

Ideally you would define sequences with an API that makes *no reference* to the specific range that a particular client cares about and then let clients pull the range they want with `take` and `drop`. This is exactly what lazy sequences provide.

Lazy Sequences

Lazy sequences are constructed using the macro `lazy-seq`:

```
(lazy-seq & body)
```

A `lazy-seq` will invoke its body only when needed, that is, when `seq` is called directly or indirectly. `lazy-seq` will then cache the result for subsequent calls. You can use `lazy-seq` to define a lazy Fibonacci series as follows:

```
src/examples/functional.clj
```

```
Line 1 (defn lazy-seq-fibo
2   ([ ]
3     (concat [0 1] (lazy-seq-fibo 0N 1N)))
4   ([a b]
5     (let [n (+ a b)]
6       (lazy-seq
7         (cons n (lazy-seq-fibo b n))))))
```

On line 3, the zero-argument body returns the concatenation of the basis values `[0 1]` and then calls the two-argument body to calculate the rest of the values. On line 5, the two-argument body calculates the next value `n` in the series, and on line 7 it conses `n` onto the rest of the values.

The key is line 6, which makes its body lazy. Without this, the recursive call to `lazy-seq-fibo` on line 7 would happen immediately, and `lazy-seq-fibo` would recurse until it blew the stack. This illustrates the general pattern: wrap the recursive part of a function body with `lazy-seq` to replace recursion with laziness.

`lazy-seq-fibo` works for small values:

```
(take 10 (lazy-seq-fibo))
-> (0 1 1N 2N 3N 5N 8N 13N 21N 34N)
```

`lazy-seq-fibo` also works for large values. Use `(rem ... 1000)` to print only the last three digits of the one millionth Fibonacci number:

```
(rem (nth (lazy-seq-fibo) 1000000) 1000)
-> 875N
```

The lazy-seq-fibo approach follows rule 3, using laziness to implement an infinite sequence. But as is often the case, you do not need to explicitly call lazy-seq yourself. By rule 5, you can reuse existing sequence library functions that return lazy sequences. Consider this use of iterate:

```
(take 5 (iterate (fn [[a b]] [b (+ a b)]) [0 1]))
-> ([0 1] [1 1] [1 2] [2 3] [3 5])
```

The iterate begins with the first pair of Fibonacci numbers: [0 1]. By working pairwise, it then calculates the Fibonacci by carrying along just enough information (two values) to calculate the next value.

The Fibonacci are simply the first value of each pair. They can be extracted by calling map first over the entire sequence, leading to the following definition of fibo suggested by Christophe Grand:

```
src/examples/functional.clj
(defn fibo []
  (map first (iterate (fn [[a b]] [b (+ a b)]) [0N 1N])))
```

fibo returns a lazy sequence because it builds on map and iterate, which also return lazy sequences. fibo is also *simple*. fibo is the shortest implementation we have seen so far. But if you are accustomed to writing imperative, looping code, correctly *choosing* fibo over other approaches may not seem simple at all! Learning to think recursively, lazily, and within the JVM's limitations on recursion—all at the same time—can be intimidating. Let the rules help you. The Fibonacci numbers are infinite: rule 3 correctly predicts that the right approach in Clojure will be a lazy sequence, and rule 5 lets the existing sequence functions do most of the work.

Lazy definitions consume *some* stack and heap. But they do not consume resources proportional to the size of an entire (possibly infinite!) sequence. Instead, *you choose* how many resources to consume when you traverse the sequence. If you want the one millionth Fibonacci number, you can get it from fibo, without having to consume stack or heap space for all the previous values.

There is no such thing as a free lunch. But with lazy sequences, you can have an infinite menu and pay only for the menu items you are eating at a given moment. When writing Clojure programs, you should prefer lazy sequences over loop/recur for any sequence that varies in size and for any large sequence.

Coming to Realization

Lazy sequences consume significant resources only as they are *realized*, that is, as a portion of the sequence is actually instantiated in memory. Clojure

works hard to be lazy and avoid realizing sequences until it is absolutely necessary. For example, `take` returns a lazy sequence and does no realization at all. You can see this by creating a var to hold, say, the first billion Fibonacci numbers:

```
(def lots-o-fibs (take 1000000000 (fibo)))
-> #'user/lots-o-fibs
```

The creation of `lots-o-fibs` returns almost immediately, because it does *almost nothing*. If you ever call a function that needs to *actually use* some values in `lots-o-fibs`, Clojure will calculate them. Even then, it will do only what is necessary. For example, the following will return the 100th Fibonacci number from `lots-o-fibs`, without calculating the millions of other numbers that `lots-o-fibs` promises to provide:

```
(nth lots-o-fibs 100)
-> 354224848179261915075N
```

Most sequence functions return lazy sequences. If you are not sure whether a function returns a lazy sequence, the function's documentation string typically will tell you the answer:

```
(doc take)
-----
clojure.core/take
([n coll])
Returns a lazy seq of the first n items in coll, or all items if
there are fewer than n.
```

The REPL, however, is *not lazy*. The printer used by the REPL will, by default, print the entirety of a collection. That is why we stuffed the first billion Fibonacci numbers into `lots-o-fibs`, instead of evaluating them at the REPL. Don't enter the following at the REPL:

```
; don't do this
(take 1000000000 (fibo))
```

If you enter the previous expression, the printer will attempt to print a billion Fibonacci numbers, realizing the entire collection as it goes. You will probably get bored and exit the REPL before Clojure runs out of memory.

As a convenience for working with lazy sequences, you can configure how many items the printer will print by setting the value of `*print-length*`:

```
(set! *print-length* 10)
-> 10
```

For collections with more than ten items, the printer will now print only the first ten followed by an ellipsis. So, you can safely print a billion fibos:

```
(take 1000000000 (fibo))
-> (0N 1N 1N 2N 3N 5N 8N 13N 21N 34N ...)
```

Or even all the fibos:

```
(fibo)
-> (0N 1N 1N 2N 3N 5N 8N 13N 21N 34N ...)
```

Lazy sequences are wonderful. They do only what is needed, and for the most part you don't have to worry about them. If you ever want to force a sequence to be fully realized, you can use either `doall` or `dorun`, discussed in [Forcing Sequences](#), on page 70.

Losing Your Head

There is one last thing to consider when working with lazy sequences. Lazy sequences let you define a large (possibly infinite) sequence and then work with a small part of that sequence in memory at a given moment. This clever ploy will fail if you (or some API) unintentionally hold a reference to the part of the sequence you no longer care about.

The most common way this can happen is if you accidentally hold the head (first item) of a sequence. In the examples in the previous sections, each variant of the Fibonacci numbers was defined as a function returning a sequence, not the sequence itself.

You could define the sequence directly as a top-level var:

```
src/examples/functional.clj
; holds the head (avoid!)
(def head-fibo (lazy-cat [0N 1N] (map + head-fibo (rest head-fibo))))
```

This definition uses `lazy-cat`, which is like `concat` except that the arguments are evaluated only when needed. This is a very pretty definition in that it defines the recursion by mapping a sum over (each element of the Fibonacci) and (each element of the *rest* of the Fibonacci).

`head-fibo` works great for small Fibonacci numbers:

```
(take 10 head-fibo)
-> (0N 1N 1N 2N 3N 5N 8N 13N 21N 34N)
```

but not so well for huge ones:

```
(nth head-fibo 1000000)
-> java.lang.OutOfMemoryError: GC overhead limit exceeded
```

The problem is that the top-level var `head-fibo` *holds the head* of the collection. This prevents the garbage collector from reclaiming elements of the sequence

after you have moved past those elements. So, any part of the Fibonacci sequence that you actually use gets cached for the life of the value referenced by `head-fibo`, which is likely to be the life of the program.

Unless you want to cache a sequence as you traverse it, you must be careful not to keep a reference to the head of the sequence. As the `head-fibo` example demonstrates, you should normally expose lazy sequences as a function that *returns* the sequence, not as a var that *contains* the sequence. If a caller of your function wants an explicit cache, the caller can always create its own var. With lazy sequences, losing your head is often a good idea.

4.3 Lazier Than Lazy

Clojure's lazy sequences are a great form of laziness at the language level. As a programmer, you can be *even lazier* by finding solutions that do not require explicit sequence manipulation at all. You can often combine existing sequence functions to solve a problem, without having to get your hands dirty at the level of `recur` or lazy sequences.

As an example of this, you will implement several solutions to the following problem.⁷ You are given a sequence of coin toss results, where heads is `:h` and tails is `:t`:

```
[ :h :t :t :h :h :h ]
```

How many times in the sequence does heads come up twice in a row? In the previous example, the answer is two. Toss 3 and toss 4 are both heads, and toss 4 and toss 5 are both heads.

The sequence of coin tosses might be very large, but it will be finite. Since you are looking for a scalar answer (a count), by rule 2 it is acceptable to use `recur`:

`src/examples/functional.clj`

```
Line 1 (defn count-heads-pairs [coll]
2   (loop [cnt 0 coll coll]
3     (if (empty? coll)
4       cnt
5       (recur (if (= :h (first coll) (second coll))
6               (inc cnt)
7               cnt)
8             (rest coll)))))
```

7. Hat tip to Jeff Brown, who posed this problem over breakfast at a No Fluff, Just Stuff symposium.

Since the purpose of the function is to count something, the loop introduces a `cnt` binding, initially zero on line 2. The loop also introduces its own binding for the `coll` so that we can shrink the `coll` each time through the `recur`. Line 3 provides the basis for the recurrence. If a sequence of coin tosses is empty, it certainly has zero runs of two heads in a row.

Line 5 is the meat of the function, incrementing the `cnt` by one if the first and second items of `coll` are both heads (`:h`).

Try a few inputs to see that `count-heads-pairs` works as advertised:

```
(count-heads-pairs [:h :h :h :t :h])
-> 2

(count-heads-pairs [:h :t :h :t :h])
-> 0
```

Although `count-heads-pairs` works, it fails as code prose. The key notion of “two in a rowness” is completely obscured by the boilerplate for `loop/recur`. To fix this, you will need to use rules 5 and 6, subdividing the problem to take advantage of Clojure’s sequence library.

The first problem you will encounter is that almost all the sequence functions do something to each element in a sequence in turn. This doesn’t help us at all, since we want to look at each element in the context of its immediate neighbors. So, let’s transform the sequence. When you see this:

```
[:h :t :t :h :h :h]
```

you should mentally translate that into a sequence of every adjacent pair:

```
[[[:h :t] [:t :t] [:t :h] [:h :h] [:h :h]]]
```

Write a function named `by-pairs` that performs this transformation. Because the output of `by-pairs` varies based on the size of its input, by rule 3 you should build this sequence lazily:

```
src/examples/functional.clj
```

```
Line 1 ; overly complex, better approaches follow...
2 (defn by-pairs [coll]
3   (let [take-pair (fn [c]
4                     (when (next c) (take 2 c)))]
5     (lazy-seq
6       (when-let [pair (seq (take-pair coll))]
7         (cons pair (by-pairs (rest coll)))))))
```

Line 3 defines a function that takes the first pair of elements from the collection. Line 5 ensures that the recursion is evaluated lazily.

Line 6 is a conditional: if the next pair does not actually contain two elements, we must be (almost) at the end of the list, and we implicitly terminate. If we do get two elements, then on line 7 we continue building the sequence by consing our pair onto the pairs to be had from the rest of the collection.

Check that `by-pairs` works:

```
(by-pairs [:h :t :t :h :h :h])
-> ((:h :t) (:t :t) (:t :h) (:h :h) (:h :h))
```

Now that you can think of the coin tosses as a sequence *of pairs* of results, it is easy to describe `count-heads-pairs` in English:

“Count the pairs of results that are all heads.”

This English description translates directly into existing sequence library functions: “Count” is `count`, of course, and “that are all heads” suggests a filter:

```
src/examples/functional.clj
(defn count-heads-pairs [coll]
  (count (filter (fn [pair] (every? #(= :h %) pair))
    (by-pairs coll))))
```

This is much more expressive than the `recur`-based implementation, and it makes clear that we are counting all the adjacent pairs of heads. But we can make things even simpler. Clojure already has a more general version of `by-pairs` named `partition`:

```
(partition size step? coll)
```

`partition` breaks a collection into chunks of size `size`. So, you could break a `heads/tails` vector into a sequence of pairs:

```
(partition 2 [:h :t :t :h :h :h])
-> ((:h :t) (:t :h) (:h :h))
```

That isn’t quite the same as `by-pairs`, which yields overlapping pairs. But `partition` can do overlaps too. The optional `step` argument determines how far `partition` moves down the collection before starting its next chunk. If not specified, `step` is the same as `size`. To make `partition` work like `by-pairs`, set `size` to 2 and set `step` to 1:

```
(partition 2 1 [:h :t :t :h :h :h])
-> ((:h :t) (:t :t) (:t :h) (:h :h) (:h :h))

(by-pairs [:h :t :t :h :h :h])
-> ((:h :t) (:t :t) (:t :h) (:h :h) (:h :h))
```

Faces of def

Throughout the book you will use various `def` forms to create vars, such as `defn`, `defmacro`, and `defmulti`. These forms are all eventually wrappers around the `def` special form.

`defonce` ensures that a var exists and sets the root binding for the var *only if it is not already set*:

```
(defonce a-symbol initial-value?)
```

`defn-` works just like `defn` but yields a *private* function that is accessible only in the namespace where it was defined.

```
(defn- name & args-as-for-defn)
```

Many other `def` forms also have dash-suffixed variants that are private.

Another possible area of improvement is the `count/filter` idiom used to count the pairs that are both heads. This combination comes up often enough that it is worth encapsulating in a `count-if` function:

```
src/examples/functional.clj
```

```
(def ^{:doc "Count items matching a filter"}
  count-if (comp count filter))
```

`comp` is used to *compose* two or more functions:

```
(comp f & fs)
```

The composed function is a new function that applies the rightmost function to its arguments, the next-rightmost function to that result, and so on. So, `count-if` will first filter and then count the results of the filter:

```
(count-if odd? [1 2 3 4 5])
-> 3
```

Finally, you can use `count-if` and `partition` to create a `count-runs` function that is more general than `count-heads-pairs`:

```
src/examples/functional.clj
```

```
(defn count-runs
  "Count runs of length n where pred is true in coll."
  [n pred coll]
  (count-if #(every? pred %) (partition n 1 coll)))
```

`count-runs` is a winning combination: both simpler and more general than the previous versions of `count-heads-pairs`. You can use it to count pairs of heads:

```
(count-runs 2 #(= % :h) [:h :t :t :h :h :h])
-> 2
```

But you can just as easily use it to count pairs of tails:

```
(count-runs 2 #(= % :t) [:h :t :t :h :h :h])
-> 1
```

Or, instead of pairs, how about runs of three heads in a row?

```
(count-runs 3 #(= % :h) [:h :t :t :h :h :h])
-> 1
```

If you still want to have a function named `count-heads-pairs`, you can implement it in terms of `count-runs`:

```
src/examples/functional.clj
(def ^{:doc "Count runs of length two that are both heads"}
  count-heads-pairs (partial count-runs 2 #(= % :h)))
```

This version of `count-heads-pairs` builds a new function using `partial`:

```
(partial f & partial-args)
```

`partial` performs a *partial application* of a function. You specify a function `f` and part of the argument list when you perform the `partial`. You specify the remainder of the argument list later, when you call the function created by `partial`. So, the following:

```
(partial count-runs 1 #(= % :h))
```

is a more expressive way of saying this:

```
(fn [coll] (count-runs 1 #(= % :h) coll))
```

Partial application is similar but not identical to *currying*.

Currying and Partial Application

When you *curry* a function, you get a new function that takes one argument and returns the original function with that one argument fixed. (Curry is named for Haskell Curry, an American logician best known for his work in combinatory logic.) If Clojure had a `curry`, it might be implemented like this:

```
; almost a curry
(defn faux-curry [& args] (apply partial partial args))
```

One use of `curry` is partial application. Here is partial application in Clojure:

```
(def add-3 (partial + 3))
(add-3 7)
-> 10
```

And here is partial application using our `faux-curry`:

```
(def add-3 ((faux-curry +) 3))
(add-3 7)
-> 10
```

If all you want is partial application, currying is just an intermediate step. Our faux-curry is not a real curry. A real curry would return a result, not a function of no arguments, once all the arguments were fixed. You can see the difference here, using the function `true?`, which takes only one argument:

```
; faux curry
((faux-curry true?) (= 1 1))
-> #<... mangled function name ...>

; if the curry were real
((curry true?) (= 1 1))
-> true
```

Since Clojure functions can have variable-length argument lists, Clojure cannot know when all the arguments are fixed. But you, the programmer, do know when you are done adding arguments. Once you have curried as many arguments as you want, just invoke the function. That amounts to adding an extra set of parentheses around the earlier expression:

```
((faux-curry true?) (= 1 1))
-> true
```

The absence of curry from Clojure is not a major problem, since `partial` is available and that is what people generally want out of curry anyway. In fact, many programmers use the terms *currying* and *partial application* interchangeably.

You have seen a lot of new forms in this section. Do not let all the details obscure the key idea: by combining existing functions from the sequence library, you were able to create a solution that was both simpler and more general than the direct approach. And, you did not have to worry about laziness or recursion at all. Instead, you worked at a higher level of abstraction and let Clojure deal with laziness and recursion for you.

4.4 Recursion Revisited

Clojure works very hard to balance the power of functional programming with the reality of the Java Virtual Machine. One example of this is the well-motivated choice of explicit TCO through `loop/recur`.

But blending the best of two worlds always runs the risk of unpleasant compromises, and it certainly makes sense to ask the question “Does Clojure

contain hidden design compromises that, while not obvious on day one, will bite me later?”

This question is *never* fully answerable for any language, but let's consider it by exploring some more complex recursions. First we will look at *mutual recursion*.

A mutual recursion occurs when the recursion bounces between two or more functions. Instead of A calls A calls A, you have A calls B calls A again. As a simple example, you could define `my-odd?` and `my-even?` using mutual recursion:

```
src/examples/functional.clj
(declare my-odd? my-even?)

(defn my-odd? [n]
  (if (= n 0)
    false
    (my-even? (dec n))))

(defn my-even? [n]
  (if (= n 0)
    true
    (my-odd? (dec n))))
```

Because `my-odd?` and `my-even?` each call the other, you need to create both vars before actually defining the functions. You could do this with `def`, but the `declare` macro lets you create both vars (with no initial binding) in a single line of code.

Verify that `my-odd?` and `my-even?` work for small values:

```
(map my-even? (range 10))
-> (true false true false true false true false true false)

(map my-odd? (range 10))
-> (false true false true false true false true false true)
```

`my-odd?` and `my-even?` consume stack frames proportional to the size of their argument, so they will fail with large numbers.

```
(my-even? (* 1000 1000 1000))
-> StackOverflowError clojure.lang.Numbers$LongOps.equiv (Numbers.java:490)
```

This is very similar to the problem that motivated the introduction of `recur`. But you cannot use `recur` to fix it, because `recur` works with self-recursion, not mutual recursion. Of course, odd/even can be implemented more efficiently *without* recursion anyway. Clojure's implementation uses bit-and (bitwise and) to implement `odd?` and `even?`:

```
; from core.clj
(defn even? [n] (zero? (bit-and n 1)))
(defn odd? [n] (not (even? n)))
```

We picked odd/even for its simplicity. Other recursive problems are not so simple and do not have an elegant nonrecursive solution. We will examine four approaches that you can use to solve such problems:

- Converting to self-recursion
- Trampolining a mutual recursion
- Replacing recursion with laziness
- Shortcutting recursion with memoization

Converting to Self-recursion

Mutual recursion is often a nice way to model separate but related concepts. For example, oddness and evenness are separate concepts but clearly related to one another.

You can convert a mutual recursion to a self-recursion by coming up with a single abstraction that deals with multiple concepts simultaneously. For example, you can think of oddness and evenness in terms of a single concept: *parity*. Define a parity function that uses `recur` and returns 0 for even numbers and 1 for odd numbers:

```
src/examples/functional.clj
(defn parity [n]
  (loop [n n par 0]
    (if (= n 0)
      par
      (recur (dec n) (- 1 par)))))
```

Test that parity works for small values:

```
(map parity (range 10))
-> (0 1 0 1 0 1 0 1 0 1)
```

At this point, you can trivially implement `my-odd?` and `my-even?` in terms of `parity`:

```
src/examples/functional.clj
(defn my-even? [n] (= 0 (parity n)))
(defn my-odd? [n] (= 1 (parity n)))
```

Parity is a straightforward concept. Unfortunately, many mutual recursions will not simplify down into an elegant self-recursion. If you try to convert a mutual recursion into a self-recursion and you find the resulting code to be

full of conditional expressions that obfuscate the definition, then do not use this approach.

Trampolining Mutual Recursion

A *trampoline* is a technique for optimizing mutual recursion. A trampoline is like an after-the-fact recur, imposed by the *caller* of a function instead of the *implementer*. Since the caller can call more than one function inside a trampoline, trampolines can optimize mutual recursion.

Clojure's trampoline function invokes one of your mutually recursive functions:

```
(trampoline f & partial-args)
```

If the return value is not a function, then a trampoline works just like calling the function directly. Try trampolining a few simple Clojure functions:

```
(trampoline list)
-> ()
(trampoline + 1 2)
-> 3
```

If the return value is a function, then trampoline assumes you want to call it recursively and calls it for you. trampoline manages its own recur, so it will keep calling your function until it stops returning functions.

Back in [Tail Recursion, on page 92](#), you implemented a tail-fibo function. You saw how the function consumed stack space and replaced the tail recursion with a recur. Now you have another option. You can take the code of tail-fibo and prepare it for trampolining by wrapping the recursive return case inside a function.

This requires adding only a single character, the #, to introduce an anonymous function:

```
src/examples/trampoline.clj
```

Line 1 ; Example only. Don't write code like this.

```
- (defn trampoline-fibo [n]
-   (let [fib (fn fib [f-2 f-1 current]
-               (let [f (+ f-2 f-1)]
-                 (if (= n current)
-                     f
-                     #(fib f-1 f (inc current))))))]
-   (cond
-     (= n 0) 0
-     (= n 1) 1
-     :else (fib 0N 1 2))))
```

The only difference between this and the original version of tail-fibo is the initial # on line 7. Try bouncing trampoline-fibo on a trampoline:

```
(trampoline trampoline-fibo 9)
-> 34N
```

Since trampoline does a recur for you, it can handle large inputs just fine, without throwing a StackOverflowError:

```
(rem (trampoline trampoline-fibo 1000000) 1000)
-> 875N
```

We have ported tail-fibo to use trampoline in order to compare and contrast trampoline and recur. For self-recursions like trampoline-fibo, trampoline offers no advantage, and you should prefer recur. But with mutual recursion, trampoline comes into its own.

Consider the mutually recursive definition of my-odd? and my-even?, which we presented at the beginning of [Section 4.4, Recursion Revisited, on page 103](#). You can convert these broken, stack-consuming implementations to use trampoline using the same approach you used to convert tail-fibo: simply prepend a # to any recursive tail calls:

```
src/examples/trampoline.clj
```

```
Line 1 (declare my-odd? my-even?)
-
- (defn my-odd? [n]
-   (if (= n 0)
5     false
-     #(my-even? (dec n))))
-
- (defn my-even? [n]
-   (if (= n 0)
10    true
-     #(my-odd? (dec n))))
```

The only difference from the original implementation is the # wrappers on lines 6 and 11. With this change in place, you can trampoline large values of n without blowing the stack:

```
(trampoline my-even? 1000000)
-> true
```

A trampoline is a special-purpose solution to a specific problem. It requires doctoring your original functions to return a different type to indicate recursion. If one of the other techniques presented here provides a more elegant implementation for a particular recursion, that is great. If not, you will be happy to have trampoline in your box of tools.

Replacing Recursion with Laziness

Of all the techniques for eliminating or optimizing recursion discussed in this chapter, laziness is the one you will probably use most often.

For our example, we will implement the `replace` function developed by Eugene Wallingford to demonstrate mutual recursion. (See <http://www.cs.uni.edu/~wallingf/patterns/recursion.html>.)

`replace` works with an *s-list* data structure, which is a list that can contain both symbols and lists of symbols. `replace` takes an *s-list*, an `oldsym`, and a `newsym` and replaces all occurrences of `oldsym` with `newsym`. For example, this call to `replace` replaces all occurrences of `b` with `a`:

```
(replace '((a b) (((b g r) (f r)) c (d e)) b) 'b 'a)
-> ((a a) (((a g r) (f r)) c (d e)) a)
```

The following is a fairly literal translation of the Scheme implementation from Wallingford's paper. We have converted from Scheme functions to Clojure functions, changed the name to `replace-symbol` to avoid collision with Clojure's `replace`, and shortened names to better fit the printed page, but we otherwise have preserved the structure of the original:

```
src/examples/wallingford.clj
; overly-literal port, do not use
(declare replace-symbol replace-symbol-expression)
(defn replace-symbol [coll oldsym newsym]
  (if (empty? coll)
      ()
      (cons (replace-symbol-expression
              (first coll) oldsym newsym)
            (replace-symbol
             (rest coll) oldsym newsym))))
(defn replace-symbol-expression [symbol-expr oldsym newsym]
  (if (symbol? symbol-expr)
      (if (= symbol-expr oldsym)
          newsym
          symbol-expr)
      (replace-symbol symbol-expr oldsym newsym)))
```

The two functions `replace-symbol` and `replace-symbol-expression` are mutually recursive, so a deeply nested structure could blow the stack. To demonstrate the problem, create a deeply-nested function that builds deeply nested lists containing a single bottom element:

```
src/examples/replace_symbol.clj
(defn deeply-nested [n]
  (loop [n n
        result '(bottom)]
```

```
(if (= n 0)
  result
  (recur (dec n) (list result))))
```

Try deeply-nested for a few small values of n:

```
(deeply-nested 5)
-> ((((((bottom))))))
```

```
(deeply-nested 25)
-> (((((((((((((((((((((((((((((((((((((((bottom))))))))))))))))))))))))))))))
```

Clojure provides a **print-level** that controls how deeply the Clojure printer will go into a nested data structure. Set the **print-level** to a modest value so that the printer doesn't go crazy trying to print a deeply nested structure. You will see that when nesting deeper, the printer simply prints a # and stops:

```
(set! *print-level* 25)
-> 25
```

```
(deeply-nested 5)
-> ((((((bottom))))))
```

```
(deeply-nested 25)
-> (((((((((((((((((((((((((((((((((((((((#))))))))))))))))))))))))))
```

Now, try to use `replace-symbol` to change `bottom` to `deepest` for different levels of nesting. You will see that large levels blow the stack. Depending on your JVM implementation, you may need a larger value than the 10000 shown here:

```
(replace-symbol (deeply-nested 5) 'bottom 'deepest)
-> ((((((deepest))))))
```

```
(replace-symbol (deeply-nested 10000) 'bottom 'deepest)
-> java.lang.StackOverflowError
```

All of the recursive calls to `replace-symbol` are inside a `cons`. To break the recursion, all you have to do is wrap the recursion with `lazy-seq`. *It's really that simple.* You can break a sequence-generating recursion by wrapping it with a `lazy-seq`. Here's the improved version. Since the transition to laziness was so simple, we could not resist the temptation to make the function more Clojurish in another way as well:

[src/examples/replace_symbol.clj](#)

```
Line 1 (defn- coll-or-scalar [x & _] (if (coll? x) :collection :scalar))
2 (defmulti replace-symbol coll-or-scalar)
3 (defmethod replace-symbol :collection [coll oldsym newsym]
4   (lazy-seq
5     (when (seq coll)
6       (cons (replace-symbol (first coll) oldsym newsym)
```

```

7      (replace-symbol (rest coll) oldsym newsym))))
8 (defmethod replace-symbol :scalar [obj oldsym newsym]
9   (if (= obj oldsym) newsym obj))

```

On line 4, the lazy-seq breaks the recursion, preventing stack overflow on deeply nested structures. The other improvement is on line 2. Rather than have two different functions to deal with symbols and lists, there is a single multimethod `replace-symbol` with one method for lists and another for symbols. (Multimethods are covered in detail in [Chapter 8, Multimethods, on page 187](#).) This gets rid of an `if` form and improves readability.

Make sure the improved `replace-symbol` can handle deep nesting:

```

(replace-symbol (deeply-nested 10000) 'bottom 'deepest)
-> ((((((((((((((((((((((((((((((((((#))))))))))))))))))))))))))

```

Laziness is a powerful ally. You can often write recursive and even mutually recursive functions and then break the recursion with laziness.

Shortcutting Recursion with Memoization

To demonstrate a more complex mutual recursion, we will look at the Hofstadter Female and Male sequences. The first Hofstadter sequences were described in *Gödel, Escher, Bach: An Eternal Golden Braid* [Hof99]. The Female and Male sequences are defined as follows:⁸

$$F(0) = 1; M(0) = 0$$

$$F(n) = n - M(F(n-1)), n > 0$$

$$M(n) = n - F(M(n-1)), n > 0$$

This suggests a straightforward definition in Clojure:

```

src/examples/male_female.clj
; do not use these directly
(declare m f)
(defn m [n]
  (if (zero? n)
    0
    (- n (f (m (dec n))))))
(defn f [n]
  (if (zero? n)
    1
    (- n (m (f (dec n))))))

```

The Clojure definition is easy to read and closely parallels the mathematical definition. However, it performs *terribly* for large values of `n`. Each value in

8. http://en.wikipedia.org/wiki/Hofstadter_sequence

the sequence requires calculating two other values from scratch, which in turn requires calculating two other values from scratch. On one of our MacBook Pro computers,⁹ it takes more than half a minute to calculate (m 250):

```
(time (m 250))
"Elapsed time: 38443.902083 msecs"
-> 155
```

Is it possible to preserve the clean, mutually recursive definition *and* have decent performance? Yes, with a little help from *memoization*. Memoization trades space for time by caching the results of past calculations. When you call a memoized function, it first checks your input against a map of previous inputs and their outputs. If it finds the input in the map, it can return the output immediately, without having to perform the calculation again.

Rebind *m* and *f* to memoized versions of themselves, using Clojure's *memoize* function:

```
src/examples/memoized_male_female.clj
(def m (memoize m))
(def f (memoize f))
```

Now Clojure needs to calculate *F* and *M* only once for each *n*. The speedup is enormous. Calculating (m 250) is thousands of times faster:

```
(time (m 250))
"Elapsed time: 5.190739 msecs"
-> 155
```

And, of course, once the memoization cache is built, “calculation” of a cached value is almost instantaneous:

```
(time (m 250))
"Elapsed time: 0.065877 msecs"
-> 155
```

Memoization alone is not enough, however. Memoization shortcuts the recursion only if the memoization cache is already populated. If you start with an empty cache and ask for *m* or *f* of a large number, you will blow the stack before the cache can be built:

```
(m 10000)
-> java.lang.StackOverflowError
```

The final trick is to guarantee that the cache is built from the ground up by exposing *sequences*, instead of *functions*. Create *m-seq* and *f-seq* by mapping *m* and *f* over the whole numbers:

9. 3.06 GHz Intel Core 2 Duo, 4 GB 667 MHz DDR2 SDRAM, Ubuntu 10.10, SSD

```
src/examples/male_female_seq.clj
```

```
(def m-seq (map m (iterate inc 0)))
(def f-seq (map f (iterate inc 0)))
```

Now callers can get $M(n)$ or $F(n)$ by taking the n th value from a sequence:

```
(nth m-seq 250)
-> 155
```

The approach is quite fast, even for larger values of n :

```
(time (nth m-seq 10000))
"Elapsed time: 0.735 msecs"
-> 6180
```

The approach we have used here is as follows:

- Define a mutually recursive function in a natural way.
- Use memoization in order to shortcut recursion for values that have already been calculated.
- Expose a sequence so that dependent values are cached before they are needed.

This approach *is* heap-consuming, in that it does cache all previously seen values. If this is a problem, you can in some situations eliminate it by selecting a more complex caching policy.

4.5 Wrapping Up

In this chapter you've seen how Clojure's support for FP strikes a well-motivated balance between academic purity and effectiveness on the Java Virtual Machine. Clojure provides a wide variety of techniques including self-recursion with recur, mutual recursion with trampoline, lazy sequences, and memoization.

Better still, for a wide variety of everyday programming tasks, you can use the sequence library, without ever having to define your own explicit recursions of lazy sequences. Functions like partition create clean, expressive solutions that are much easier to write.

If Clojure were an exclusively FP language, we would turn our attention next to the challenges of living in a world without mutable state and perhaps begin discussing the state monad. But Clojure leads us in a different direction with its most innovative feature: explicit APIs for managing mutable state. Clojure reference model provides four different semantics to model state and is what we'll tackle in [Chapter 5, State, on page 113](#).